Geavanceerde computerarchitecturen - Sessie 4

Cas Truyers en Niels Verhoeve

26 maart 2026

1 Introductie

In dit labo kijken we naar optimalisatietechnieken voor geheugenoperaties op de GPU. Hiervoor deden we onderzoek naar de verschillende methoden voor geheugenopslag waarna we dit verder implementeerden. In een eerste deel kijken we naar de invloed van het verwerken van de arrays in een verschillende volgorde. In een tweede deel kijken we naar de invloed van verschillende geheugentypes. Voor dit labo beperken we ons tot vierkante matrices, N rijen en kolommen, met eenvoudige nummers (0-9) voor de eenvoud. Uiteraard zijn de conclusies uitbreidbaar voor matrices van verschillende grote. De volledige code is te vinden op: https://github.com/Niels-V-KU/Lab_Geavanceerde_Computerarchitectuur/tree/main.

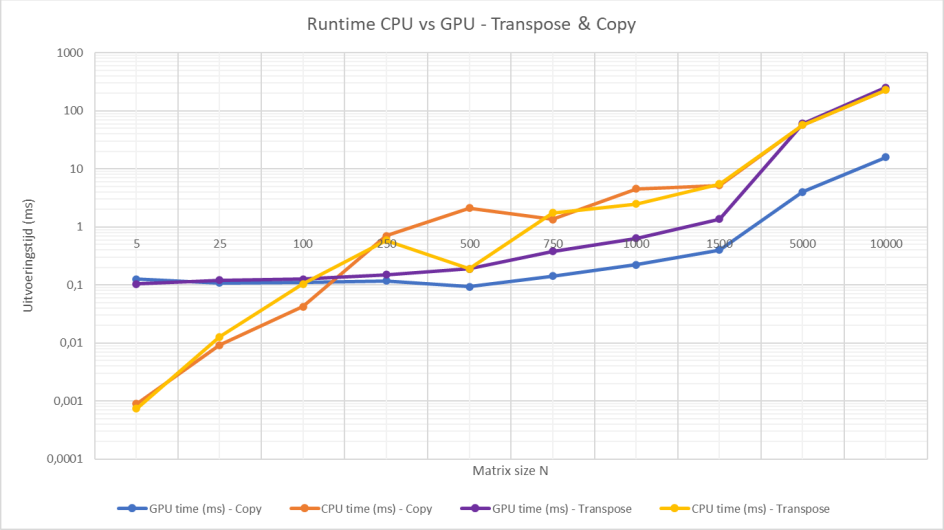
2 Deel 1 - Coalesced memory accesss

```
__global__ void copyArray(float *a, float *b, int N) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x*gridDim.x;
    for(int i = idx; i<N*N; i+=stride){
        if (i < N*N) {
            b[i] = a[i];
        }
    }
}

__global__ void transposeArray(float *a, float *b, int N) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x*gridDim.x;
    for(int i = idx; i<N*N; i+=stride){
        if (i < N*N) {
            int row = i / N;
            int col = i % N;
            b[col * N + row] = a[row * N + col];
        }
    }
}
```

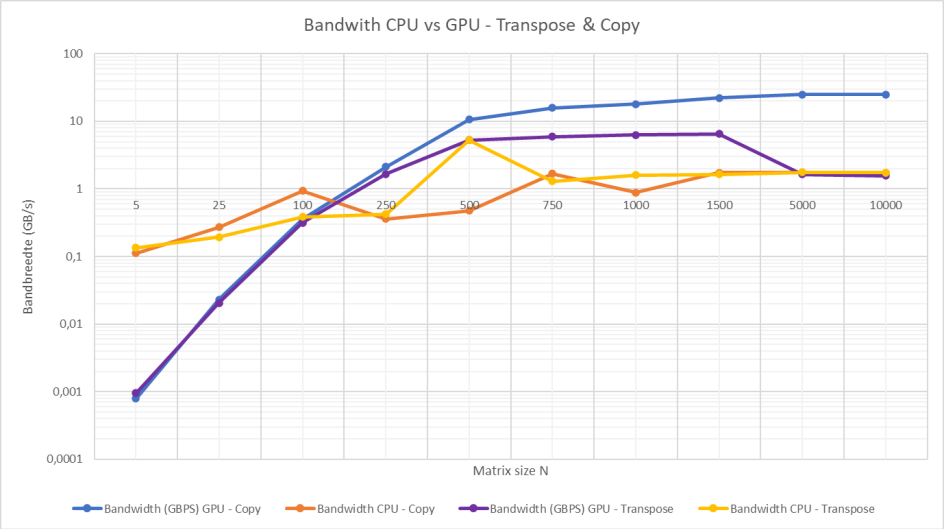
Voor deel 1 starten we met het kopiëren en transponeren van een matrix. In eerste instantie verwachten we met de GPU een zekere tijdswinst te halen voor het transformeren van de matrices, aangezien we deze bewerkingen in parallelle operaties kunnen opdelen. Hiervoor werd de originele 2D-matrix opgeslagen als een aaneengesloten 1D-array. Dit garandeert dat alle array elementen altijd worden opgeslagen in aaneengesloten geheugenlocaties, en is belangrijk voor de verdere werking van onze code.

Voor de copy kernel kunnen dan alle elementen in aaneengesloten adressen geselecteerd en gekopieerd worden via group memory access (coalesced memory accesss). Voor de transpose kernel moet er meer gesprongen worden binnen deze geheugenadressen (rijen naar kolommen), waardoor we tragere geheugenoperaties verwachten. De geheugenoperaties zijn niet coalesced wat leid tot meerdere geheugenoperaties per warp.



Figuur 1: Runtime Transpose & copy

Om dit te controleren meten we de uitvoeringstijd en berekenen hieruit de bandwidth (BW). Uit figuur 1 kunnen we dan zien dat de uitvoeringstijd van de GPU ongeveer constant is voor kleine arrays, dit suggereert een minimum uitvoeringstijd door overhead. We halen bijgevolg ook geen profijt uit het gebruik van de GPU voor kleine arrays ($N \leq 100$). De uitvoeringstijd van de GPU ten opzichte van de CPU groeit echter geleidelijker voor in grote toenemende arrays. Voor grote arrays ($N \geq 5000$) zien we ook een sprong in de grafiek, waarbij er opnieuw een minimaal verschil is tussen CPU en GPU uitvoeringstijd. Dit is veroorzaakt door een bottleneck in geheugenoperaties, we zullen dit verder proberen oplossen.



Figuur 2: bandwidth Copy en Transpose kernels

Dezelfde conclusies zijn af te leiden uit figuur 2. De bandwidth (BW), oftewel de data transfer rate, neemt sterker toe bij de GPU voor groeiende matrix sizes (met een bepaalde verzadiging). De CPU bandwidth neemt minder sterk toe voor toenemende sizes, er is dus een optimalisatie mogelijk via GPU processing. We zien ook opnieuw dat voor de transfer kernel voor veel elementen de GPU geen winst meer haalt.

3 Deel 2 - Global, shared, and constant memory

In het vorige deel zien we dat de geheugenoperaties beperkend zijn voor de tijdswinst haalbaar met de GPU. We kunnen dit oplossen aan de hand van verschillende soorten geheugen die we in dit deel verder zullen bekijken.

3.1 Basis

Voor dit deel vermenigvuldigen we twee matrixen in verschillende vormen van geheugen. We kozen er hierbij voor om verder te werken met contiguous 1D-arrays. We zetten twee basemarks aan de hand van een CPU implementatie, en een GPU implementatie met enkel global memory. Vervolgens maken we nog twee kernels met shared en constant memory geïncludeerd om de uitvoering van de geheugenoperaties te versnellen.

Ons eerdere conclusies uit hoofdstuk 2 worden hier opnieuw bevestigd, zie figuur 3. Er word bevestigd dat voor grotere arrays de GPU uitvoeringstijd kleiner is dan deze voor de CPU. Voor kleine arrays is de CPU vaak nog sneller en zien we een zekere constante uitvoeringstijd voor de GPU, dit toont het sterke parallellisme aan van de GPU.

```
    // global
__global__ void multiplyArrays(float *a, float *b, float*c, int N) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x*gridDim.x;
    for(int i = idx; i<N*N; i+=stride){
        int row = i / N;
        int col = i % N;
        for (int e = 0; e < N; e++){
            c[i] += a[row*N + e] * b[N*e + col];
        }
    }
}
    // global + shared
#define N 100
__global__ void multiplyArrays(float *a, float *b, float*c) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x*gridDim.x;
    __shared__ float temp_c[N*N];

    for(int i = idx; i<N*N; i+=stride){
        int row = i / N;
        int col = i % N;
        temp_c[i] = 0.0f;
        for (int e = 0; e < N; e++){
            temp_c[i] += a[row*N + e] * b[N*e + col];
        }
        c[i] = temp_c[i];
    }
}
    // global + shared + constant
#define N 100
__constant__ float const_a[N*N];
__constant__ float const_b[N*N];
__global__ void multiplyArrays(float*c, float tot_time) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x*gridDim.x;

    __shared__ float tempc[N*N];

    for(int i = idx; i<N*N; i+=stride){
        int row = i / N;
        int col = i % N;
        for (int e = 0; e < N; e++){
            tempc[i] += const_a[row*N + e] * const_b[N*e + col];
        }

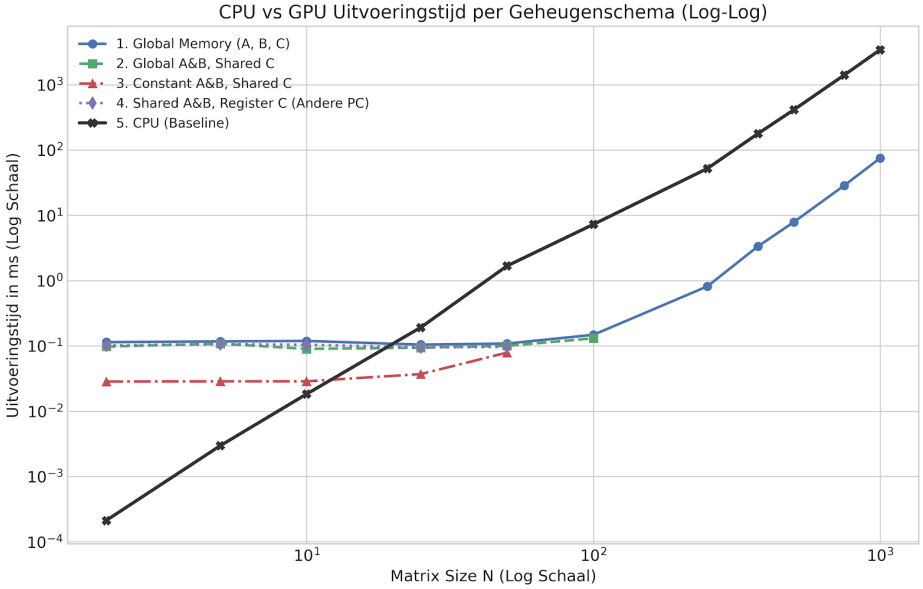
        c[i] = tempc[i];
    }
}
    // global + shared memory and registers
```

```
#define N 100
__global__ void multiplyArrays(float *a, float *b, float*c) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x*gridDim.x;
    __shared__ float temp_a[N*N];
    __shared__ float temp_b[N*N];

    for(int i = idx; i<N*N; i+=stride){
        temp_a[i] = a[i];
        temp_b[i] = b[i];
    }
    __syncthreads();

    for(int i = idx; i<N*N; i+=stride){
        int row = i / N;
        int col = i % N;
        float sum = 0.0f;
        for (int e = 0; e < N; e++){
            sum += temp_a[row*N + e] * temp_b[N*e + col];
        }
        c[i] = sum;
    }
}
```

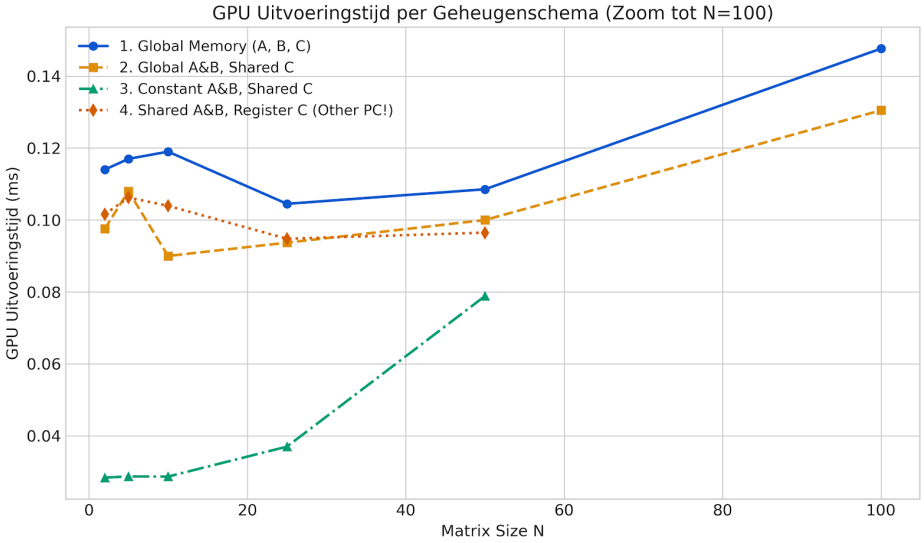
In bovenstaande code bevind zich dezelfde kernel een aantal maal herschreven voor het experimenteren met de verschillende soorten geheugens, elk getest in hun eigen progamma. __syncthreads() zorgt er hier voor dat alle threads binnen een block hun shared memory writes hebben voltooid (Dit komt ook terug in de verbeterde code, zie 3.2).



Figuur 3: Memory Scheme met CPU

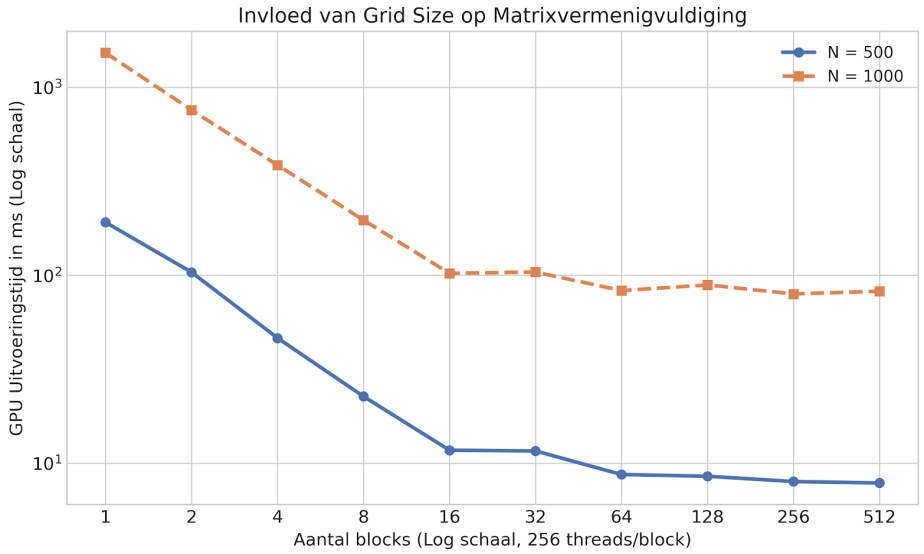
Vervolgens zoomen we in op enkel de GPU technieken. Global memory was sneller dan de CPU implementatie (bij voldoende grote arrays) maar is de traagste van de GPU geheugens, dit is zoals verwacht aangezien global memory zich off-chip bevind. De snelste methode is deze met shared en constant memory, ook dit is volgens onze voorspelling, zie figuur 4.

We moeten hier echter opletten met de grote van ons snelle geheugen. Voor grotere arrays kunnen we niet de volledige array rechtstreeks schrijven naar het snelle geheugen, maar moeten de operaties opgesplitst worden om een geheugen overload te voorkomen.



Figuur 4: Invloed memory scheme

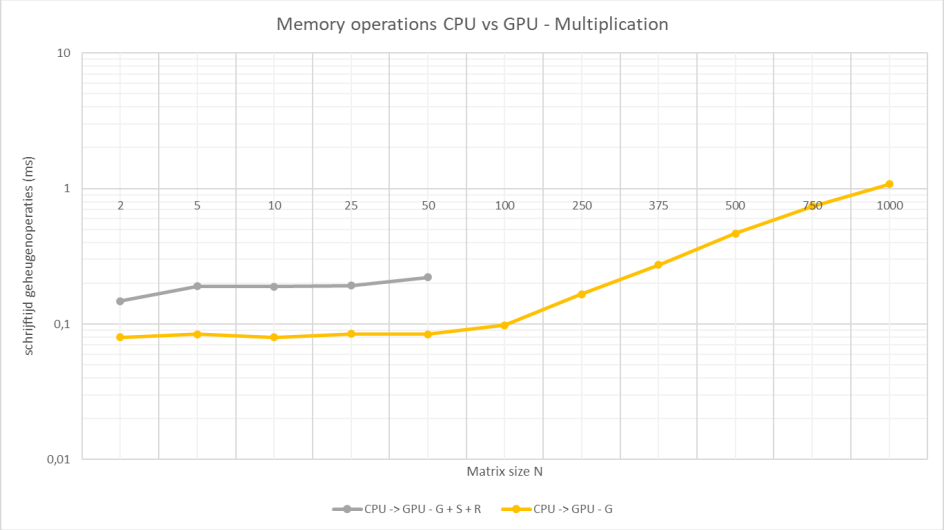
Bij het testen met verschillende grid sizes zien we een lagere uitvoeringstijd bij toenemende grote (voor gelijke arrays). Ook hier is echter een zekere verzadiging, bij ongeveer 16 blocks met 256 threads/block. Deze trend blijft ook constant voor arrays verschillend in grote, zie figuur 5.



Figuur 5: Invloed van gridsize

De verschillende schrijfoperaties vragen ook een bepaalde tijd. Dit is zichtbaar in figuur 6. We maken hierbij een vergelijking tussen de benchmark met global memory en de de oplossing met shared en constant geheugen. We merken hierbij op dat we enkel de geheugenoperaties in de main code loop meten (schrijven van en naar gpu geheugen en niet de geheugenoperaties binnen de kernels.

We kunnen concluderen dat door het gebruik van verschillende soorten geheugens we ons programma significant kunnen versnellen. We zien echter wel dat schrijven naar verschillende soorten geheugens meer tijd vraagt dan alles schrijven naar global memory, alhoewel het verschil relatief klein is (beide operaties duren minder lang dan een ms). We gaan hier echter wel uit van een betere situatie dan de praktijk waarbij er ook schrijfoperaties gebeuren in de kernel, en waarbij we de trend van global- en shared geheugen doortrekken voor verdere vergelijking met enkel global memory. We zitten hier nog met het probleem van memory overflow. We zullen dit nog oplossen, zie hoofdstuk 3.2.



Figuur 6: Geheugenoperaties

3.2 Verbeteringen

We merkten dat er nog enkele verbeteringen mogelijk waren in de eerder geschreven code, om deze reden eindigen we met een kernel die gebruikmaakt van tiled shared memory. Dit lost het eerdere probleem op dat ons snelle geheugen overloopt voor grote payloads.

```
#define N 5000 // N kan ook meegegeven worden als argument.
#define BLOCK_SIZE 16
#define CUDA_CHECK(x) if((x)!=cudaSuccess){printf("CUDA error\n"); exit(1);}

// functions
__global__ void multiplyArrays(float *A, float *B, float *C)
{
    __shared__ float As[BLOCK_SIZE][BLOCK_SIZE];
    __shared__ float Bs[BLOCK_SIZE][BLOCK_SIZE];

    int row = blockIdx.y * BLOCK_SIZE + threadIdx.y;
    int col = blockIdx.x * BLOCK_SIZE + threadIdx.x;

    float value = 0;

    for (int t = 0; t < (N + BLOCK_SIZE - 1) / BLOCK_SIZE; t++)
    {
        int tiledCol = t * BLOCK_SIZE + threadIdx.x;
        int tiledRow = t * BLOCK_SIZE + threadIdx.y;

        if (row < N && tiledCol < N)
            As[threadIdx.y][threadIdx.x] = A[row * N + tiledCol];
        else
            As[threadIdx.y][threadIdx.x] = 0;

        if (col < N && tiledRow < N)
            Bs[threadIdx.y][threadIdx.x] = B[tiledRow * N + col];
        else
            Bs[threadIdx.y][threadIdx.x] = 0;

        __syncthreads();

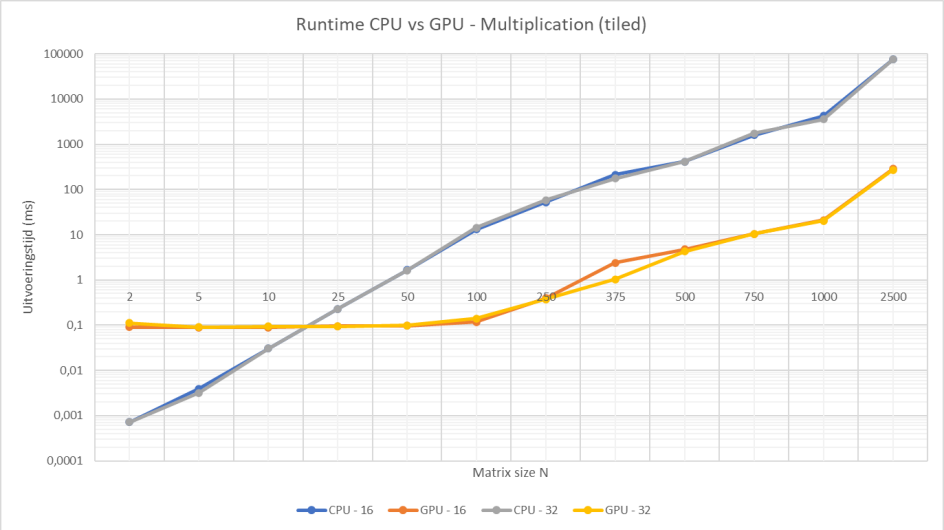
        for (int k = 0; k < BLOCK_SIZE && (t * BLOCK_SIZE + k) < N; k++)
            value += As[threadIdx.y][k] * Bs[k][threadIdx.x];

        __syncthreads();
    }

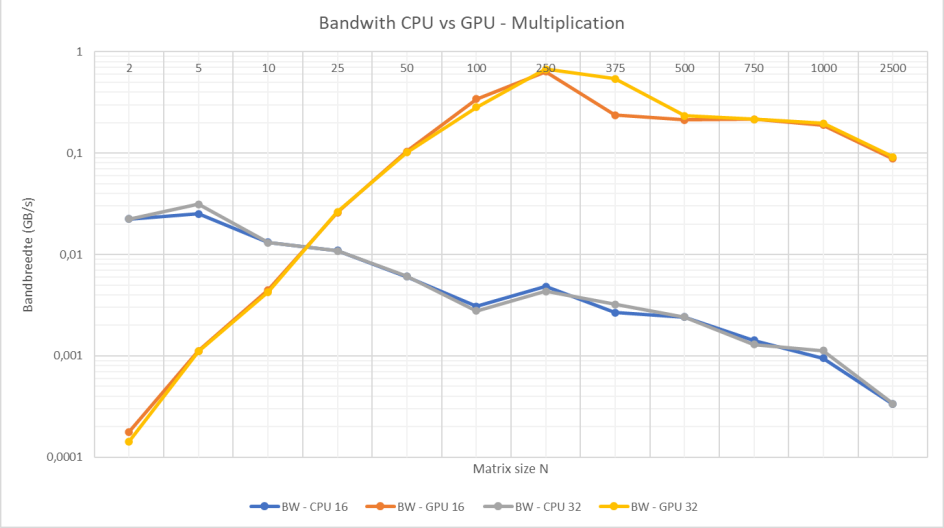
    if (row < N && col < N)
        C[row * N + col] = value;
```

```
41 }
```

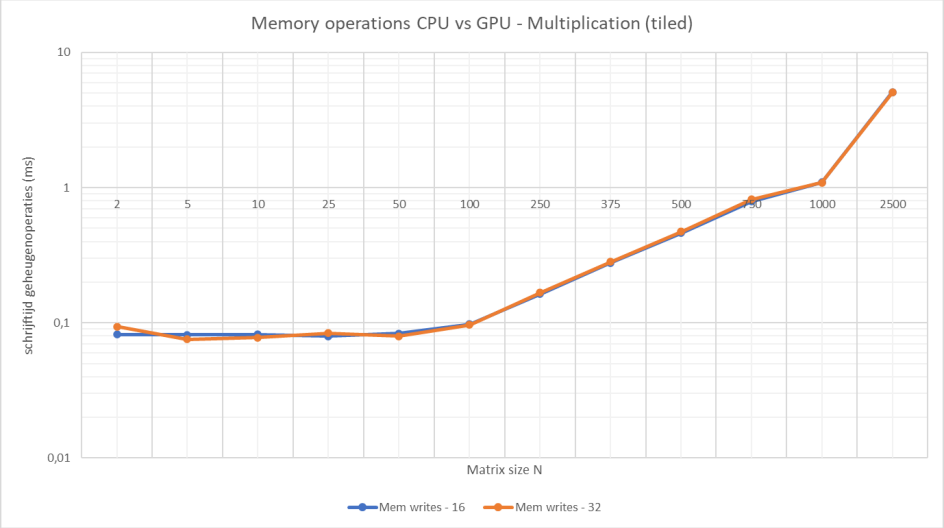
We bevestigen ons eerdere conclusies en veronderstellingen. Met het probleem van memory overflow opgelost kunnen we gebruik maken van de voordelen van snel geheugen, waarbij we deze ook kunnen gebruiken en vergelijken voor grotere arrays. Wat we bijkomend zien is dat voor heel grote arrays de tijd voor geheugen schrijfoperaties sterk toeneemt. Ook zien we hier dat de bandwidth nog steeds een zeker optimum heeft voor middelgrote arrays, maar dankzij de toegevoegde verbeteringen blijven we ook voor grotere arrays een snelheidswinst behouden ten opzichte van de CPU benchmark. Dit wordt duidelijk uit onderstaande figuren.



Figuur 7: Runtime voor tiled memory



Figuur 8: bandwidth voor tiled memory



Figuur 9: Geheugenoperaties voor tiled memory

4 Conclusie

We merken dat er veel factoren een invloed hebben op de snelheid van ons programma. Bijvoorbeeld het gebruikte geheugen en de volgorde waarin data wordt opgeslagen. Door hierin bedachtzame keuzes te maken kunnen we voor grote data, met een sterk parallel karakter, de uitvoering van ons programma significant versnellen.